Corso di laurea in Informatica (L-31)

Algoritmi e Strutture Dati

Heap e code con priorità

Dall'albero completo a una struttura dati compatta ed efficiente

---

Simone Faro

Indice

1 Quando non basta rispettare l'ordine di arrivo **2**
1.1 Il pronto soccorso come modello intuitivo . . . 2
1.2 Dall'ospedale ai sistemi informatici . . . 2
1.3 La coda con priorità come tipo di dato astratto . . . 3

2 L'heap binario: un albero con due regole **4**
2.1 La forma prima dei valori . . . 4
2.2 La proprietà di priorità . . . 4
2.3 Un heap non è un albero binario di ricerca . . . 5

3 Far vivere l'heap come un albero **5**
3.1 Heapify: rimettere ordine dall'alto verso il basso . . . 6
3.2 Inserimento: occupare l'ultima posizione e risalire . . . 7
3.3 Estrazione della priorità massima o minima . . . 7
3.4 Quando cambia una chiave: Increase-Key e Decrease-Key . . . 8
3.5 Eliminare un nodo arbitrario . . . 9
3.6 La lezione dell'invariante . . . 9

4 L'albero diventa un array **9**
4.1 Memorizzazione per livelli . . . 9
4.2 Le formule per padre e figli . . . 10
4.3 Un piccolo incontro con le operazioni bitwise . . . 10
4.4 La distinzione tra dimensione e capacità . . . 11

5 Operazioni sul max-heap rappresentato con un array **11**
5.1 Consultare la priorità massima . . . 12
5.2 Heapify-Max: la versione su array . . . 12
5.3 Risali-Max e inserimento . . . 13
5.4 Estrai-Max: rimuovere l'elemento di priorità massima . . . 13
5.5 Increase-Key e Decrease-Key nel max-heap . . . 14
5.6 Eliminazione di un elemento noto . . . 15
5.7 Costruire un max-heap da un array qualunque . . . 15
5.8 La sorpresa: costruire un heap costa tempo lineare . . . 16

6 Il min-heap: la stessa struttura, un ordine rovesciato **17**
6.1 Heapify-Min e la risalita . . . 17
6.2 Inserire un record ed estrarre il minimo . . . 18
6.3 Decrease-Key e Increase-Key nel min-heap . . . 19
6.4 Eliminazione e costruzione per un min-heap . . . 19
6.5 Un'unica idea dietro due famiglie di procedure . . . 20

7 Quanto costano davvero le operazioni? **20**
7.1 Un confronto con le liste, senza dimenticare le ipotesi . . . 21
7.2 Aggiornare un record conoscendone soltanto l'identificatore . . . 21
7.3 Priorità uguali: ordine di arrivo e stabilità . . . 22

8 Tre applicazioni che fanno emergere la struttura **22**
8.1 Una simulazione: il futuro in una coda . . . 22
8.2 Cammini minimi: quando una distanza migliora . . . 22
8.3 Huffman: costruire un codice scegliendo due minimi . . . 23

9 Una breve storia: l'idea, la forma, l'efficienza **23**

10 Conclusioni: quanto ordine serve davvero? **23**

1 Quando non basta rispettare l'ordine di arrivo

Ci sono code nelle quali la regola è semplice: chi arriva per primo viene servito per primo. È la logica *first in, first out* (FIFO) di una coda ordinaria, perfettamente ragionevole per una fila alla cassa o per le richieste che devono essere elaborate nell'ordine di ricezione. Ma la regola dell'arrivo non è sempre quella giusta. Immaginiamo un pronto soccorso in cui una persona con un problema lieve sia arrivata alcuni minuti prima di una persona in condizioni critiche: trattarle come due richieste indistinguibili sarebbe una scelta evidentemente inadeguata.

Il punto non è costruire una fila più complicata; è cambiare la *domanda* a cui la struttura dati deve rispondere. Una coda ordinaria chiede "chi è arrivato prima?". Una *coda con priorità* chiede invece "chi deve essere servito adesso?". Il criterio può essere l'urgenza clinica, la scadenza di un evento, il costo di un percorso o il peso di un simbolo. In ciascun caso un insieme di elementi cambia nel tempo e deve essere possibile trovare rapidamente quello più importante secondo una regola prefissata.

1.1 Il pronto soccorso come modello intuitivo

Per fissare le idee, associamo a ogni richiesta un numero intero, tanto maggiore quanto più urgente è la richiesta. Consideriamo questa piccola sequenza, *puramente illustrativa* e non coincidente con un protocollo clinico reale:

Richiesta	Ordine d'arrivo	Priorità	Significato nell'esempio
Anna	1	2	Valutazione che può attendere
Bruno	2	5	Intervento estremamente urgente
Carla	3	4	Intervento urgente
Diego	4	5	Stessa priorità assegnata a Bruno

Se tutte e quattro le richieste sono ancora in attesa, una coda con priorità massima servirà anzitutto Bruno, poi Diego, quindi Carla e infine Anna, *a condizione* di aver stabilito che, a parità di priorità, prevalga chi è arrivato prima. Questo ultimo dettaglio è importante: il numero di priorità determina solo un ordinamento parziale dei casi in presenza di valori uguali. Per specificare completamente il comportamento dobbiamo stabilire una regola per gli *ex aequo*, ad esempio usando come chiave composta (priorità, −ordine di arrivo) quando si seleziona il massimo.

Nella pratica ospedaliera il triage è naturalmente più articolato: le categorie hanno significato clinico, le condizioni dei pazienti possono cambiare e la valutazione non si riduce a un numero immutabile. Proprio quest'ultima osservazione anticipa un'operazione essenziale della nostra struttura dati: *modificare la priorità di una richiesta già inserita*.

Il collegamento ha anche una storia affascinante. Il termine francese *triage*, da *trier* (separare, scegliere), fu associato all'organizzazione dei soccorsi in ambito militare durante l'epoca napoleonica. Al chirurgo Dominique-Jean Larrey, attivo tra la fine del Settecento e l'inizio dell'Ottocento, vengono ricondotte importanti innovazioni nel trattamento tempestivo dei feriti; gli storici discutono tuttavia le attribuzioni esclusive e il ruolo di altri medici, tra cui Pierre-François Percy [6, 7]. Qui ci interessa la domanda che attraversa i secoli: *come decidere in modo efficace a chi dedicare per primi risorse limitate?*

1.2 Dall'ospedale ai sistemi informatici

La stessa struttura astratta compare in contesti molto diversi. In un sistema operativo, un pianificatore può dare precedenza a processi o attività in base a livelli di priorità, scadenze o politiche di servizio; aumentare la priorità di un processo significa modificare il suo posto nell'ordine di scelta. In una simulazione a

eventi discreti, invece, dobbiamo elaborare per primo l'evento con il *tempo previsto più piccolo*: una coda di priorità minima consente di estrarlo, eseguirlo e inserire gli eventuali nuovi eventi che ne derivano.

Negli algoritmi sui grafi la priorità può essere una distanza. Nel metodo per i cammini minimi associato a Edsger W. Dijkstra, pubblicato nel 1959 [2], è naturale selezionare di volta in volta il vertice con distanza provvisoria minima e aggiornare le distanze quando troviamo cammini migliori. Un'altra applicazione istruttiva è la costruzione dei codici di Huffman, introdotti da David A. Huffman nel 1952: a ogni passo vengono selezionati i due pesi più piccoli per formare un nuovo nodo [4]. In entrambi i casi una coda con priorità minima è una scelta implementativa particolarmente naturale.

Abbiamo quindi tre esempi con una proprietà comune: le richieste *arrivano*, una viene *selezionata ed estratta*, e qualche priorità può *cambiare*. Non abbiamo bisogno di mantenere tutti gli elementi ordinati: ci basta sapere quale elemento deve essere estratto e aggiornare efficacemente la struttura quando qualcosa cambia.

1.3 La coda con priorità come tipo di dato astratto

Definizione 1.1 (Coda con priorità)**.** Una *coda con priorità* è una collezione dinamica di record dotati di una chiave di priorità. Una *max-priority queue* permette di consultare ed estrarre un record con chiave massima; una *min-priority queue* permette di consultare ed estrarre un record con chiave minima. Inserimento, aggiornamento della priorità e interrogazione del numero di elementi completano l'interfaccia fondamentale.

Le operazioni che useremo sono INSERISCI, MASSIMO o MINIMO, ESTRAIMASSIMO o ESTRAIMINIMO, AUMENTACHIAVE, DIMINUISCICHIAVE, DIMENSIONE e VUOTA. Le prime due interrogazioni restituiscono il record di priorità estrema *senza eliminarlo*; le operazioni di estrazione invece restituiscono quel record *e lo rimuovono*. In alcuni programmi è utile anche ELIMINA, che cancella un elemento specifico senza attendere che raggiunga la priorità estrema, e COSTRUISCI, che inizializza la struttura a partire da molti record.

Questa definizione descrive *che cosa* deve fare la struttura dati, non *come* realizzarla. Una semplice lista non ordinata consentirebbe inserimenti in tempo costante, ma obbligherebbe a scandire fino a n elementi per trovare la priorità massima o minima. Una lista ordinata renderebbe immediata la selezione, ma potrebbe richiedere tempo lineare per ogni inserimento o aggiornamento. Vorremmo ottenere un compromesso migliore: selezione immediata e modifiche veloci, anche quando la collezione è molto grande.

Tabella 1: Confronto orientativo tra implementazioni di una coda con priorità di n elementi. Per la lista ordinata si assume una rappresentazione che consenta l'accesso immediato all'estremo desiderato.

Rappresentazione	Inserimento	Consultazione estremo	Estrazione estremo
Lista non ordinata	$O(1)$	$O(n)$	$O(n)$
Lista ordinata	$O(n)$	$O(1)$	$O(1)$
Heap binario	$O(\log n)$	$O(1)$	$O(\log n)$

La terza riga anticipa la struttura che studieremo: l'*heap binario*. Non è una coda già ordinata e non è un albero di ricerca. È un modo molto più economico di custodire soltanto le relazioni d'ordine indispensabili.

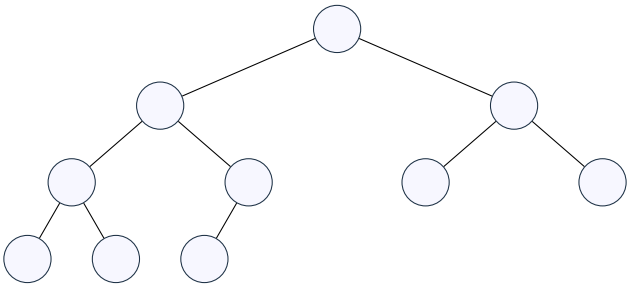
2 L'heap binario: un albero con due regole

2.1 La forma prima dei valori

Cominciamo dimenticando per un momento le priorità. Un heap binario ha la forma di un *albero binario completo*: tutti i livelli, tranne eventualmente l'ultimo, sono completamente occupati; le posizioni dell'ultimo livello sono riempite da sinistra verso destra, senza buchi. Alcuni testi chiamano questa forma anche *quasi completa*, per distinguerla da un albero in cui anche l'ultimo livello è pieno.

Definizione 2.1 (Forma di un heap binario)**.** Un heap binario di n nodi è un albero binario radicato, riempito per livelli da sinistra a destra. Ogni nodo ha al più due figli; l'unico possibile livello incompleto è l'ultimo, le cui posizioni occupate formano un prefisso da sinistra.

La forma non dipende dalle chiavi. Un heap con dieci elementi e un altro heap con dieci elementi hanno quindi *esattamente la stessa forma*; possono differire solo per i valori presenti nei nodi. Aggiungere un elemento significa occupare la prossima posizione libera in fondo, non scegliere un punto dell'albero in base alla grandezza della nuova chiave.



Ultimo livello occupato da sinistra, senza posizioni saltate

Figura 1: La forma di un albero binario completo con dieci nodi. Per il momento i nodi sono privi di chiavi: la disposizione dipende soltanto dal numero di elementi.

Questa regolarità è tutt'altro che un dettaglio estetico. Se i livelli crescono raddoppiando il numero massimo di nodi, non può esserci un cammino dalla radice a una foglia arbitrariamente lungo: la profondità resta logaritmica. Inoltre la sequenza dei nodi letti *per livelli* non contiene buchi e potrà essere conservata direttamente in un array. Quest'ultima conseguenza sarà il cuore della seconda parte della dispensa.

Proposizione 2.2 (Altezza dell'heap)**.** Un heap binario non vuoto con n nodi ha altezza

$$h = \lfloor \log_2 n \rfloor ,$$

dove l'altezza è il numero di archi del cammino più lungo dalla radice a una foglia.

Dimostrazione. I livelli da 0 a $h-1$ sono pieni e contengono $1 + 2 + \cdots + 2^{h-1} = 2^h - 1$ nodi. Poiché esiste almeno un nodo al livello h, abbiamo $n \geq 2^h$. D'altra parte tutti i nodi si trovano nei livelli da 0 a h, che possono contenere al più $2^{h+1} - 1$ nodi: dunque $n < 2^{h+1}$. Ne segue $h \leq \log_2 n < h + 1$, ossia $h = \lfloor \log_2 n \rfloor$. □

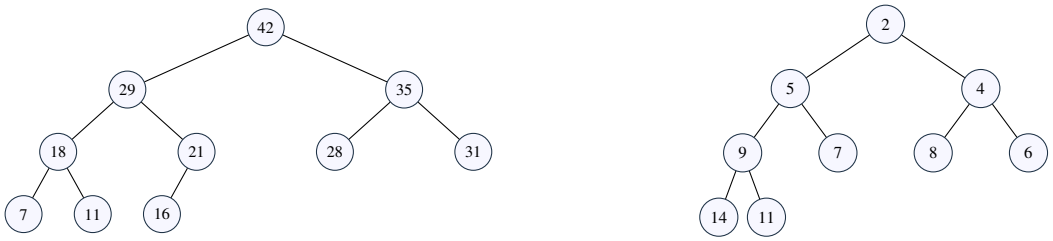
2.2 La proprietà di priorità

A questo punto associamo una chiave a ogni nodo. La seconda regola è sorprendentemente locale: non chiediamo che tutto l'albero sia ordinato, ma che un padre rispetti una relazione con ciascuno dei figli. Se ci interessa la priorità *massima*, ogni padre deve avere una chiave almeno grande quanto quelle dei figli. Se ci interessa la priorità *minima*, la relazione viene rovesciata.

Definizione 2.3 (Max-heap e min-heap)**.** Un *max-heap* è un heap binario in cui per ogni nodo u e ogni suo figlio v vale

$$\text{chiave}(u) \geq \text{chiave}(v).$$

In un *min-heap* vale invece $\text{chiave}(u) \leq \text{chiave}(v)$ per ogni coppia padre–figlio. Le chiavi uguali sono ammesse.



Max-heap: la radice contiene 42 Min-heap: la radice contiene 2

Figura 2: Due heap binari: l'ordine è imposto lungo gli archi, non tra i nodi di uno stesso livello.

Perché una condizione tanto debole è sufficiente? Consideriamo un qualunque nodo v del max-heap e risaliamo lungo il cammino fino alla radice r. A ogni passo la chiave non diminuisce: il padre è almeno grande quanto il figlio. Per transitività, $\text{chiave}(r) \geq \text{chiave}(v)$. La radice contiene dunque *una* chiave massima dell'intera struttura. Per il min-heap lo stesso ragionamento, con le disuguaglianze rovesciate, prova che la radice contiene una chiave minima.

Proposizione 2.4 (La radice è l'elemento estremo)**.** La radice di un max-heap contiene un elemento di priorità massima. La radice di un min-heap contiene un elemento di priorità minima.

Dimostrazione. La proprietà locale confronta la chiave di ciascun nodo con quella dei suoi figli. Applicandola lungo il cammino dalla radice a un nodo arbitrario otteniamo, per transitività, la relazione desiderata tra la chiave della radice e la chiave del nodo considerato. □

2.3 Un heap non è un albero binario di ricerca

Il nodo 29 nel max-heap della figura ha un fratello di valore 35: pur essendo a sinistra, non è maggiore di quello a destra. Questo non è un errore. In un albero binario di ricerca le posizioni a sinistra e a destra sono vincolate dalla relazione tra le *chiavi di interi sottoalberi*. In un heap il vincolo riguarda *soltanto padre e figli*; fra i due sottoalberi di uno stesso nodo non imponiamo alcun ordine. Cercare un valore arbitrario, quindi, non è in generale veloce: può essere necessario ispezionare $\Theta(n)$ elementi.

Il vantaggio è un altro. A noi interessa soprattutto l'estremo, non la posizione di qualsiasi chiave. Mantenendo una proprietà locale meno esigente, possiamo aggiornare l'heap con pochi scambi lungo un solo cammino. È un esempio importante di progettazione di strutture dati: *non memorizzare più informazioni di quelle necessarie alle operazioni che vogliamo rendere efficienti*.

3 Far vivere l'heap come un albero

La proprietà dell'heap è facile da verificare; il problema interessante è conservarla quando la collezione cambia. Supponiamo di avere un max-heap corretto e di modificare un solo nodo. In quale parte dell'albero può essersi prodotto un errore? La risposta dipende dalla direzione della modifica. Se una chiave *aumenta*, può diventare troppo grande rispetto al padre; se *diminuisce*, può diventare troppo piccola

rispetto ai figli. Da questa osservazione nascono due movimenti fondamentali: la *risalita verso la radice* e l'operazione HEAPIFY, che ripristina la proprietà in una direzione discendente.

3.1 Heapify: rimettere ordine dall'alto verso il basso

Immaginiamo che il max-heap della figura precedente conservi la sua forma, ma che alla radice, al posto di 42, sia stato collocato il valore 16. I due sottoalberi radicati nei figli continuano a soddisfare la proprietà di max-heap; è la radice a essere troppo piccola. Non sarebbe necessario ricostruire tutto da capo. Basta confrontare il valore 16 con quelli dei figli, 29 e 35. Il figlio più grande è 35: lo facciamo salire e lasciamo scendere 16 al suo posto. Ora il problema si ripresenta eventualmente più in basso, lungo un solo ramo.

Definizione 3.1 (Heapify per un max-heap)**.** HEAPIFY-MAX riceve un nodo u i cui sottoalberi sinistro e destro sono già max-heap. Ripristina la proprietà di max-heap nel sottoalbero radicato in u, spostando eventualmente verso il basso il record che inizialmente occupava quel nodo.

Algoritmo 1 HEAPIFY-MAX-NODO(u): versione concettuale sull'albero

Dati: Il figlio sinistro e il figlio destro di u, se presenti, sono radici di max-heap

```
mentre u possiede almeno un figlio fai
    v ← figlio di u con chiave massima
    se chiave(u) ≥ chiave(v) allora
        restituisci
    fine se
    scambia(record(u), record(v))
    u ← v
fine mentre
```

La scelta del figlio maggiore non è arbitraria. Se scambiassimo il nodo corrente con il figlio minore, potremmo lasciare al suo posto un valore ancora più piccolo dell'altro figlio. Promuovendo il maggiore, invece, rendiamo subito corretta la relazione tra il nodo corrente e *entrambi* i figli. Il vecchio record può creare un problema soltanto nella posizione in cui è sceso. Ripetiamo lo stesso ragionamento finché il record non raggiunge un punto in cui nessun figlio possiede una chiave maggiore, oppure una foglia.

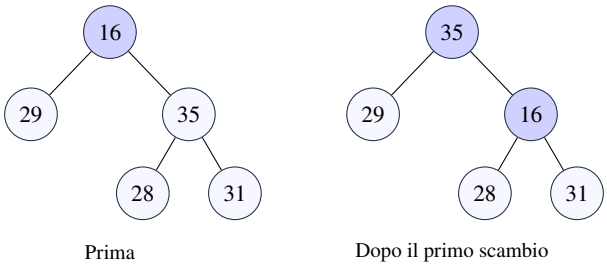


Figura 3: L'idea di HEAPIFY-MAX: il valore 16 viene scambiato prima con 35 e poi, proseguendo nel sottoalbero destro, con 31. Sono rappresentati soltanto i nodi coinvolti e i loro vicini.

Proposizione 3.2 (Correttezza di Heapify)**.** Se i due sottoalberi di u sono max-heap prima della chiamata, HEAPIFY-MAX-NODO(u) trasforma l'intero sottoalbero di u in un max-heap, senza cambiarne la forma né i record contenuti.

Dimostrazione. Se il record in u è almeno grande quanto entrambi i figli, la proprietà è già soddisfatta in u e, per ipotesi, in tutti i nodi sottostanti. Altrimenti il maggiore fra i figli viene scambiato con u: il record

promosso domina il figlio non scelto, mentre i due sottoalberi conservano tutti i propri record. L'unica relazione che può essere diventata scorretta è quella fra il record sceso e i suoi nuovi figli. La procedura ripete la stessa operazione in quel punto, scendendo di un livello a ogni scambio. Poiché l'albero è finito, il procedimento termina e tutte le relazioni padre–figlio risultano corrette. □

Per un min-heap il ragionamento è identico, ma scegliamo il *figlio di chiave minima* e scambiamo quando il padre è maggiore di quel figlio. Denomineremo questa procedura HEAPIFY-MIN. In entrambi i casi il percorso visita al massimo un nodo per livello: con un heap di n elementi il costo è $O(\log n)$ e lo spazio ausiliario è $O(1)$ se l'operazione è realizzata iterativamente.

3.2 Inserimento: occupare l'ultima posizione e risalire

Supponiamo di inserire una nuova richiesta di priorità 36 nel max-heap con radice 42. Prima di decidere come disporre i valori, dobbiamo conservare la forma dell'albero: il nuovo nodo viene collocato *nella prima posizione libera dell'ultimo livello, procedendo da sinistra a destra*; se il livello è completo, inauguriamo il livello successivo.

L'aggiunta di una foglia non altera le relazioni di priorità fra i vecchi nodi. Può però violare la relazione tra il nuovo nodo e suo padre. Nell'esempio 36 è inserito sotto il nodo 21: per un max-heap è troppo grande. Scambiamo i due record, ottenendo 36 sopra 21. Ma ora 36 si trova sotto 29, che è ancora troppo piccolo: scambiamo di nuovo. Infine 36 incontra 42, che lo domina, e il procedimento si arresta.

Algoritmo 2 RISALI-MAX-NODO(u): riparare un possibile errore verso il padre

```
1: mentre u non è la radice e chiave(u) > chiave(padre(u)) fai
2:     p ← padre(u)
3:     scambia(record(u), record(p))
4:     u ← p
5: fine mentre
```

Per inserire un elemento nel max-heap, dunque, basta creare l'ultima foglia e applicare RISALI-MAX-NODO a essa. Ogni scambio corregge la relazione con un padre, e nessun altro ramo viene modificato. Il costo è $O(\log n)$, perché il nodo può raggiungere al più la radice. Nel min-heap la risalita avviene, con la stessa logica, quando il nuovo valore è *minore* di quello del padre.

L'inserimento chiarisce anche una distinzione spesso trascurata: *la posizione del nuovo nodo è fissata dalla forma; la posizione finale del nuovo record è determinata dalle chiavi*. Perciò non abbiamo un albero di ricerca, in cui il percorso di inserimento dipende dall'ordine dei valori: qui il nodo nasce sempre nell'ultima posizione e soltanto dopo i record possono scambiarsi.

3.3 Estrazione della priorità massima o minima

Consultare la priorità estrema è immediato: il record cercato si trova alla radice. Più impegnativa è l'*estrazione*, perché dobbiamo rimuovere il record e lasciare un heap valido. Eliminare direttamente la radice creerebbe un vuoto nella parte più alta dell'albero, violando la regola della forma completa. L'intuizione risolutiva è utilizzare l'*ultima foglia*, l'unico nodo che possiamo rimuovere senza creare un buco.

Per estrarre il massimo da un max-heap di almeno due nodi compiamo quattro azioni: memorizziamo il record della radice, spostiamo alla radice il record dell'ultima foglia, eliminiamo quella foglia e richiamiamo HEAPIFY-MAX sulla radice. La forma rimane completa; poiché i due sottoalberi della radice erano già max-heap, HEAPIFY-MAX possiede esattamente la precondizione di cui ha bisogno.

Riprendiamo il nostro heap di dieci elementi. Il massimo è 42 e l'ultima foglia contiene 16. Dopo la sostituzione, l'albero ha nove nodi e 16 alla radice. Confrontando 16 con 29 e 35, poi con 28 e 31, HEAPIFY-MAX lo fa scendere di due livelli. La radice finale contiene 35, il nuovo massimo. L'insieme degli elementi è quello iniziale privato del solo 42.

Per un min-heap estraiamo, simmetricamente, il record minimo dalla radice e applichiamo HEAPIFY-MIN dopo aver spostato in alto il record dell'ultima foglia. Se l'heap ha un solo elemento, l'estrazione lo restituisce e lascia una struttura vuota: nessun HEAPIFY è necessario. Su un heap vuoto la consultazione o l'estrazione sono operazioni non definite, che il nostro pseudocodice segnalerà come errore.

Il costo. Localizzare l'ultima foglia nella forma completa può essere fatto seguendo il percorso determinato dall'indice dell'ultimo nodo, in tempo $O(\log n)$ in una rappresentazione esplicita ad albero. La successiva discesa di HEAPIFY costa anch'essa $O(\log n)$. Più avanti, con un array, l'ultima foglia sarà disponibile in tempo costante e il costo complessivo resterà $O(\log n)$, con costanti ridotte e senza puntatori ai figli.

3.4 Quando cambia una chiave: Increase-Key e Decrease-Key

Torniamo al pronto soccorso. Una persona in attesa può essere rivalutata: la sua priorità può crescere, se il caso diventa più urgente, oppure diminuire, se le condizioni consentono di attendere. La modifica non implica che la richiesta venga estratta e reinserita: possiamo intervenire direttamente sul record già presente nell'heap, *purché sappiamo dove si trovi*.

In un *max-heap*, aumentare la chiave di un nodo non può renderlo più piccolo dei suoi figli: può invece renderlo troppo grande per il padre. Di conseguenza AUMENTACHIAVE-MAX modifica la chiave e applica la risalita. Diminuire una chiave produce l'effetto opposto: non può violare la relazione con il padre, ma può renderla troppo piccola rispetto ai figli. Perciò DIMINUISCICHIAVE-MAX richiama HEAPIFY-MAX sul nodo modificato.

Nel *min-heap* le direzioni si invertono. Diminuire una chiave può violare la relazione con il padre e richiede una risalita. Aumentarla può violare la relazione con i figli e richiede HEAPIFY-MIN. La tabella riassume tutte e quattro le possibilità.

Tabella 2: La direzione in cui può propagarsi una violazione dopo la modifica di una chiave.

Struttura	AUMENTACHIAVE	DIMINUISCICHIAVE
Max-heap	Risalita verso il padre	HEAPIFY-MAX verso i figli
Min-heap	HEAPIFY-MIN verso i figli	Risalita verso il padre

Per esempio, nel min-heap della figura possiamo abbassare la chiave della foglia 11 a 1. Questo valore viene dapprima scambiato con 9, poi con 5 e infine con 2, fino a diventare la nuova radice. Dunque non è la procedura di modifica in sé a essere complessa: il problema si riduce alla scelta della *direzione corretta*. Tanto la risalita quanto HEAPIFY attraversano al più un cammino dalla radice a una foglia e richiedono $O(\log n)$ tempo.

C'è però una precisazione algoritmica essenziale. Scrivere "modifica la chiave del record x" non significa che localizzare x sia gratuito. Se abbiamo soltanto un heap senza informazioni ausiliarie, trovare un record specifico può richiedere $O(n)$ tempo. I limiti $O(\log n)$ per AUMENTACHIAVE e DIMINUISCICHIAVE presuppongono che la posizione del record sia già nota, oppure che venga mantenuto un identificatore capace di localizzarlo rapidamente. Riprenderemo il punto quando l'heap sarà memorizzato in un array.

3.5 Eliminare un nodo arbitrario

Supponiamo che una richiesta venga cancellata prima del suo turno. Togliere un nodo interno lascerebbe un buco; anche qui possiamo rimpiazzarlo con il record dell'ultima foglia e rimuovere quest'ultima. Dopo la sostituzione, però, il nuovo record potrebbe essere troppo grande per il padre *oppure* troppo piccolo per i figli. In un max-heap scegliamo quindi la risalita se il nuovo valore supera quello del padre, altrimenti applichiamo HEAPIFY-MAX. Nel min-heap invertiamo la direzione del confronto. Poiché nessuna delle due riparazioni attraversa più di un cammino, la cancellazione di un nodo *già individuato* costa $O(\log n)$.

3.6 La lezione dell'invariante

Tutte le operazioni precedenti sembrano diverse, ma condividono un'unica struttura di ragionamento. Partiamo da un heap corretto; produciamo una modifica estremamente localizzata; osserviamo dove la proprietà padre–figlio potrebbe risultare falsa; eseguiamo scambi soltanto lungo il cammino in cui la violazione può propagarsi. Non è necessario riorganizzare l'intera collezione perché la struttura conserva una quantità di ordine sufficiente a rendere locale il lavoro di riparazione.

La *forma completa* garantisce la profondità logaritmica; la *proprietà di priorità* garantisce che tutti i problemi possano essere ricondotti a una sola direzione del percorso. Le due regole collaborano, ma svolgono ruoli differenti: senza la prima il percorso potrebbe essere lungo n; senza la seconda la radice non avrebbe alcuna ragione di contenere l'elemento estremo.

4 L'albero diventa un array

Finora abbiamo ragionato in termini di nodi, padri e figli. È stato utile perché la forma geometrica mostra immediatamente come funzionano inserimenti e riparazioni. Ma una rappresentazione letterale con puntatori non è necessaria. La forma completa dell'heap contiene infatti un'informazione sorprendente: *possiamo ricostruire tutti i collegamenti dell'albero conoscendo soltanto la posizione di un elemento in un array*.

4.1 Memorizzazione per livelli

Disponiamo gli elementi dell'albero nell'array A in *ordine di livello*: prima la radice, poi i due nodi del livello successivo da sinistra a destra, poi i quattro nodi del livello seguente, e così via. Se il nostro max-heap contiene dieci record, otteniamo

$$A = [\,42,\ 29,\ 35,\ 18,\ 21,\ 28,\ 31,\ 7,\ 11,\ 16\,].$$

La prima cella dell'array è la radice; le celle successive corrispondono a tutti i posti occupati dell'albero, senza celle vuote intermedie. L'array non è ordinato in senso crescente o decrescente, e non deve esserlo. Contiene esattamente le relazioni di priorità imposte dall'heap.

Poiché ogni nodo è identificato da un indice, non occorre memorizzare puntatori a padre, figlio sinistro e figlio destro. Il costo per conservare l'heap è quello degli n record, più la dimensione della collezione e, se l'array è dinamico, la capacità eventualmente disponibile. Le relazioni fra nodi non occupano memoria aggiuntiva.

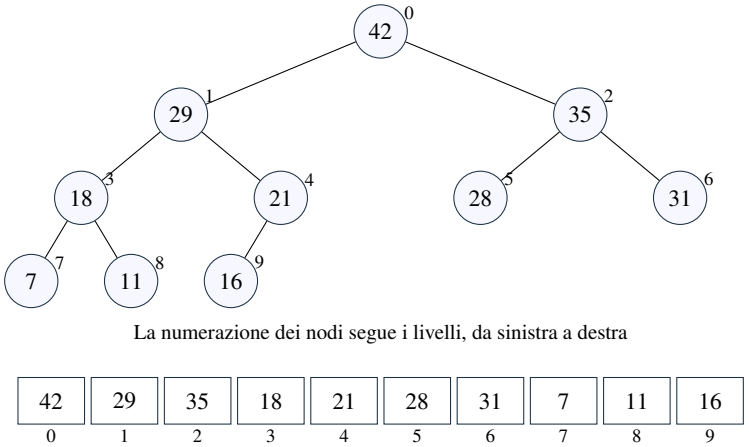


Figura 4: La stessa struttura in due forme: albero completo (in alto) e array (in basso). Gli indici in piccolo identificano le medesime posizioni nelle due rappresentazioni.

4.2 Le formule per padre e figli

D'ora in poi gli array sono indicizzati a partire da 0, come nella dispensa sull'ordinamento. Per il nodo di indice i definiamo

$$\text{Left}(i) = 2i + 1, \qquad \text{Right}(i) = 2i + 2, \qquad \text{Parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor \quad (i > 0).$$

Il figlio sinistro esiste soltanto se $2i + 1 < n$; quello destro esiste soltanto se $2i + 2 < n$. La radice ha indice 0 e non possiede un padre. Queste condizioni non sono particolari tecnici secondari: un accesso a un figlio con indice oltre $n - 1$ è un accesso a una cella *non appartenente* all'heap, anche se l'array ha capacità maggiore di n.

Perché proprio queste formule? Il livello d occupa, quando è pieno, gli indici da $2^d - 1$ a $2^{d+1} - 2$. I due figli di un nodo si trovano nel livello successivo e sono disposti consecutivamente, con quello sinistro prima del destro. Seguendo questa numerazione per livelli, il nodo i ha figli $2i + 1$ e $2i + 2$. Invertendo la relazione, sia un figlio sinistro $j = 2i + 1$ sia un figlio destro $j = 2i + 2$ riconducono allo stesso padre $i = \lfloor (j - 1)/2 \rfloor$.

Nell'esempio, il nodo in $A[4]$ contiene 21; il suo padre è in $A[\lfloor (4 - 1)/2 \rfloor] = A[1]$ e vale 29. Il suo figlio sinistro è in $A[9]$ e vale 16, mentre il figlio destro avrebbe indice 10 e quindi non esiste, perché $n = 10$. La possibilità di calcolare tali posizioni con poche operazioni aritmetiche è una conseguenza diretta del riempimento senza buchi.

Possiamo anche ricavare immediatamente quali nodi siano foglie. Tutti gli indici $i \geq \lfloor n/2 \rfloor$ non hanno figli, perché $2i + 1 \geq n$. Di conseguenza, gli indici dei nodi interni sono precisamente $0, \ldots, \lfloor n/2 \rfloor - 1$. Questa osservazione diventerà determinante per costruire un heap partendo da un array qualsiasi.

4.3 Un piccolo incontro con le operazioni bitwise

Un computer memorizza gli indici in forma binaria. Spostare verso sinistra di un bit un intero non negativo equivale a moltiplicarlo per due, *purché il risultato sia rappresentabile nel tipo usato*. Possiamo

dunque scrivere le relazioni di parentela anche come

$$\begin{aligned} \text{Left}(i) &= (i \ll 1) + 1 = (i \ll 1) \mid 1, \\ \text{Right}(i) &= (i + 1) \ll 1, \\ \text{Parent}(i) &= (i - 1) \gg 1 \quad (i > 0). \end{aligned}$$

Qui $\ll$ indica lo spostamento binario a sinistra, $\gg$ quello a destra e $\mid$ l'OR bit a bit. L'identità per il figlio sinistro funziona perché $i \ll 1$ termina sempre con un bit 0: sommare 1 e applicare l'OR con 1 producono lo stesso risultato. La formula per il padre utilizza uno spostamento a destra su un intero non negativo, equivalente alla divisione intera per due.

Per esempio, se $i = 5$, la sua rappresentazione binaria è 101_2. Il figlio sinistro ha indice $1011_2 = 11$ e il destro $1100_2 = 12$; entrambi hanno padre di indice 5. La corrispondenza è esatta, non una tecnica approssimativa.

Tabella 3: Formule aritmetiche e versioni bitwise equivalenti, con indici non negativi e senza overflow.

Operazione	Espressione aritmetica	Espressione bitwise
Figlio sinistro	$2i + 1$	$(i \ll 1) \mid 1$
Figlio destro	$2i + 2$	$(i + 1) \ll 1$
Padre ($i > 0$)	$\lfloor (i - 1)/2 \rfloor$	$(i - 1) \gg 1$

Questa è un'ottima occasione per capire che la rappresentazione binaria degli interi può essere sfruttata per esprimere elegantemente la struttura di un albero. Non bisogna tuttavia dedurne automaticamente che uno shift sia oggi più veloce di una moltiplicazione per due: i compilatori moderni sono normalmente capaci di produrre istruzioni equivalenti. In un codice reale contano innanzitutto leggibilità, controlli sui limiti, assenza di overflow e prestazioni misurate. Dal punto di vista dell'analisi asintotica, tutte queste formule richiedono $O(1)$ tempo nel modello a parole di macchina.

4.4 La distinzione tra dimensione e capacità

Quando scriviamo un heap come $A[0..n-1]$, n è il numero di record *attualmente presenti*. Un array fisico può avere una capacità $C \geq n$. Le celle da $A[n]$ ad $A[C-1]$ sono riservate ma non appartengono all'heap; quindi non partecipano ai confronti e non devono comparire in HEAPIFY.

Per inserire un record, scriviamo in $A[n]$ soltanto se esiste spazio libero, poi portiamo la dimensione a $n + 1$. Se la capacità è esaurita, un array dinamico può allocare spazio maggiore, spesso raddoppiandolo, e copiare i record. Questo ridimensionamento costa occasionalmente $O(n)$ tempo; con il raddoppio geometrico l'inserimento mantiene comunque il costo *ammortizzato* $O(\log n)$. Per tenere nitido lo pseudocodice supporremo, dove necessario, che l'array abbia già capacità sufficiente, segnalando esplicitamente le operazioni che richiedono un eventuale ridimensionamento.

5 Operazioni sul max-heap rappresentato con un array

La seconda parte può ora essere letta come la traduzione operativa della prima. Non introduciamo una nuova struttura dati: *è sempre lo stesso albero*, ma i suoi nodi sono le celle consecutive dell'array. Per ogni procedura useremo A per l'array e n per il numero di elementi effettivamente presenti. Aggiungeremo i parametri indispensabili per l'operazione richiesta, come l'indice i, una nuova chiave k o un record x. Quando un'operazione cambia la dimensione, restituiremo anche il nuovo valore di n.

Il confronto fra record verrà indicato, per brevità, con $A[i] > A[j]$ e $A[i] < A[j]$, intendendo il confronto fra le rispettive *chiavi*. Se le priorità sono uguali, lo pseudocodice può scegliere uno qualunque dei record equivalenti, salvo che si sia definita una chiave composta per risolvere i pareggi.

5.1 Consultare la priorità massima

Nel max-heap la prima operazione è quasi troppo semplice per meritare un algoritmo: leggere $A[0]$. Proprio la sua semplicità, però, spiega perché siamo disposti a sostenere qualche scambio in più durante gli aggiornamenti. Ogni operazione di manutenzione mantiene una promessa: *il record di priorità massima sarà sempre a portata di mano, nella prima cella*.

Algoritmo 3 MASSIMO(A, n): consultazione senza rimozione

```
1: se n = 0 allora
2:     errore: heap vuoto
3: fine se
4: restituisci A[0]
```

Il costo temporale è $\Theta(1)$, lo spazio ausiliario è $\Theta(1)$ e l'heap non viene modificato. Per conoscere la dimensione è sufficiente leggere n; per verificare se la struttura è vuota basta confrontare n con zero.

5.2 Heapify-Max: la versione su array

Nel codice non abbiamo bisogno di chiedere a un nodo quali siano i suoi figli. Basta calcolare $2i + 1$ e $2i + 2$, facendo attenzione ai limiti dell'heap. Una variabile m conserva l'indice dell'elemento più grande tra il nodo corrente e i figli esistenti. Se $m = i$, nessuno scambio è necessario; altrimenti facciamo salire $A[m]$, lasciamo scendere $A[i]$ e proseguiamo dalla nuova posizione.

Algoritmo 4 HEAPIFY-MAX(A, n, i): ripristino discendente della proprietà di max-heap

```
Dati: 0 ≤ i < n; i sottoalberi radicati nei figli di i sono max-heap
1: mentre vero fai
2:     m ← i
3:     ℓ ← 2i + 1; r ← 2i + 2
4:     se ℓ < n e A[ℓ] > A[m] allora
5:         m ← ℓ
6:     fine se
7:     se r < n e A[r] > A[m] allora
8:         m ← r
9:     fine se
10:    se m = i allora
11:        restituisci
12:    fine se
13:    scambia(A[i], A[m])
14:    i ← m
15: fine mentre
```

È importante osservare che il confronto con il figlio destro usa $A[m]$, non necessariamente $A[i]$. Potremmo avere già scoperto che il figlio sinistro è più grande del padre: a quel punto dobbiamo verificare se il figlio destro sia ancora più grande. Soltanto così scegliamo sempre il massimo fra i tre candidati.

Se l'altezza del sottoalbero di radice i è h, ogni iterazione richiede un numero costante di operazioni e scende al più di un livello, quindi il tempo è $O(h+1)$, in particolare $O(\log n)$. La procedura è iterativa e usa poche variabili: lo spazio ausiliario è $O(1)$.

5.3 Risali-Max e inserimento

Per realizzare la risalita basta sostituire il puntatore al padre con $\lfloor (i-1)/2 \rfloor$. Non c'è nessuna ricerca nell'albero: l'indice del padre è determinato aritmeticamente. Se il record appena inserito vale più del padre, scambiamo i due record e ripetiamo; se non lo supera, l'heap è di nuovo corretto.

Algoritmo 5 RISALI-MAX(A, i): riparazione verso la radice

```
1: mentre i > 0 fai
2:     p ← ⌊(i − 1)/2⌋
3:     se A[p] ≥ A[i] allora
4:         restituisci
5:     fine se
6:     scambia(A[p], A[i])
7:     i ← p
8: fine mentre
```

Algoritmo 6 INSERISCI-MAX(A, n, x): inserimento di un record x

Dati: L'array ha una cella disponibile in posizione n

```
1: A[n] ← x
2: RISALI-MAX(A, n)
3: restituisci n + 1
```

Eseguiamo un inserimento completo. Partiamo da

$$A = [\,42, 29, 35, 18, 21, 28, 31, 7, 11, 16\,], \qquad n = 10.$$

Inseriamo il valore 36 nella posizione $A[10]$. Il suo padre si trova in $A[4]$ e contiene 21, quindi scambiamo 36 con 21. La nuova posizione è $i = 4$, il cui padre è $A[1] = 29$: scambiamo ancora. In $i = 1$ troviamo un padre $A[0] = 42$, che è maggiore di 36, e ci fermiamo. Il risultato è

$$A = [\,42, 36, 35, 18, 29, 28, 31, 7, 11, 16, 21\,], \qquad n = 11.$$

Notiamo che l'inserimento non ha richiesto una scansione dell'array né il riordino completo dei valori. Due scambi sono stati sufficienti. Il costo peggiore è $O(\log n)$, lo spazio ausiliario $O(1)$, senza considerare un eventuale ridimensionamento dell'array.

5.4 Estrai-Max: rimuovere l'elemento di priorità massima

L'array consente di identificare immediatamente l'ultima foglia: è $A[n-1]$. L'operazione di estrazione può quindi essere espressa con poche istruzioni. Conserviamo $A[0]$ in una variabile, copiamo $A[n-1]$ in $A[0]$, riduciamo n e ripristiniamo la proprietà con HEAPIFY-MAX.

Algoritmo 7 ESTRAI-MAX(A, n): estrazione e aggiornamento della dimensione

```
1: se n = 0 allora
2:     errore: heap vuoto
3: fine se
4: x ← A[0]
5: se n = 1 allora
6:     restituisci (x, 0)
7: fine se
8: A[0] ← A[n − 1]
9: n ← n − 1
10: HEAPIFY-MAX(A, n, 0)
11: restituisci (x, n)
```

Per seguire tutti i movimenti, riprendiamo l'heap iniziale da dieci elementi. Prima dell'estrazione abbiamo

$$[\,42, 29, 35, 18, 21, 28, 31, 7, 11, 16\,].$$

La variabile x conserva 42. Il valore 16 dell'ultima foglia prende il posto della radice e la dimensione diventa 9. HEAPIFY-MAX confronta 16 con 29 e 35, poi con 28 e 31. La successione è la seguente:

$$\begin{aligned}[\,\mathbf{16}, 29, 35, 18, 21, 28, 31, 7, 11\,] &\longrightarrow [\,35, 29, \mathbf{16}, 18, 21, 28, 31, 7, 11\,] \\ &\longrightarrow [\,35, 29, 31, 18, 21, 28, \mathbf{16}, 7, 11\,].\end{aligned}$$

Al termine, 42 è stato restituito e non appartiene più all'heap; il nuovo massimo 35 è alla radice. Tutte le altre chiavi compaiono esattamente una volta. L'estrazione richiede $O(\log n)$ tempo nel caso peggiore e $O(1)$ spazio ausiliario.

5.5 Increase-Key e Decrease-Key nel max-heap

Le due procedure sembrano quasi speculari, ma richiamano operazioni ausiliarie diverse. AUMENTA-CHIAVE-MAX richiede che la nuova chiave k non sia inferiore alla precedente; dopo la sostituzione, l'unico conflitto possibile è con il padre. DIMINUISCI-CHIAVE-MAX richiede invece che k non sia superiore alla precedente; il conflitto può trovarsi con uno dei figli.

Algoritmo 8 AUMENTA-CHIAVE-MAX(A, n, i, k)

```
Dati: 0 ≤ i < n
1: se k < chiave(A[i]) allora
2:     errore: non è un aumento di chiave
3: fine se
4: chiave(A[i]) ← k
5: RISALI-MAX(A, i)
```

Algoritmo 9 DIMINUISCI-CHIAVE-MAX(A, n, i, k)

```
Dati: 0 ≤ i < n
1: se k > chiave(A[i]) allora
2:     errore: non è una diminuzione di chiave
3: fine se
4: chiave(A[i]) ← k
5: HEAPIFY-MAX(A, n, i)
```

Per esempio, partendo da $[42, 29, 35, 18, 21, 28, 31, 7, 11, 16]$, DIMINUISCI-CHIAVE-MAX$(A, 10, 1, 4)$ cambia 29 in 4. Il valore 4 scende prima sotto 21 e poi sotto 16. Il risultato è

$$[\,42, 21, 35, 18, 16, 28, 31, 7, 11, 4\,].$$

Il record resta presente; è cambiata soltanto la sua priorità e, per conservare l'invariante, la sua posizione nell'heap. Se conosciamo i, entrambe le procedure richiedono $O(\log n)$ tempo e $O(1)$ spazio aggiuntivo.

5.6 Eliminazione di un elemento noto

Consideriamo un record già individuato in posizione i. Possiamo sovrascriverlo con l'ultimo record dell'array e ridurre la dimensione. Se il nuovo record è più grande del padre, la violazione può essere soltanto verso l'alto e usiamo la risalita. Altrimenti, se c'è un problema, sarà verso i figli e ricorriamo a HEAPIFY-MAX.

Algoritmo 10 ELIMINA-MAX(A, n, i): eliminazione per indice

Dati: $0 \le i < n$

```
1: x ← A[i]
2: se i = n − 1 allora
3:     restituisci (x, n − 1)
4: fine se
5: A[i] ← A[n − 1]
6: n ← n − 1
7: se i > 0 e A[i] > A[⌊(i − 1)/2⌋] allora
8:     RISALI-MAX(A, i)
9: altrimenti
10:    HEAPIFY-MAX(A, n, i)
11: fine se
12: restituisci (x, n)
```

Anche qui il tempo è $O(\log n)$, ma va ribadita la precondizione: *l'indice dell'elemento da eliminare deve essere già noto*. Cercare prima il record in un heap qualsiasi può rendere l'operazione complessivamente lineare.

5.7 Costruire un max-heap da un array qualunque

Finora abbiamo immaginato di costruire l'heap inserendo gli elementi uno per uno. Se partiamo da un array arbitrario di n valori, potremmo effettivamente inserire ogni valore in una struttura inizialmente vuota; poiché ogni inserimento richiede al più $O(\log n)$ tempo, otterremmo un algoritmo di costruzione da $O(n \log n)$ tempo.

Esiste però un'idea migliore. Tutte le foglie di un albero sono già heap: non hanno figli con cui violare la proprietà. Il primo nodo che può richiedere un intervento è quindi *l'ultimo nodo interno*; una volta riparato il suo sottoalbero, possiamo passare al nodo interno precedente. Procedendo all'indietro negli indici, quando raggiungiamo un nodo i suoi figli radicano già heap validi. La precondizione di HEAPIFY-MAX è dunque sempre soddisfatta.

Algoritmo 11 COSTRUISCI-MAX-HEAP(A, n): costruzione dal basso

```
1: per i ← ⌊n/2⌋ − 1 fino a 0, decrescendo fai
2:     HEAPIFY-MAX(A, n, i)
3: fine per
```

Ecco un esempio concreto. Prendiamo l'array inizialmente non ordinato

$$[\,4, 10, 3, 5, 1, 8, 6\,].$$

I nodi interni hanno indici $2, 1, 0$. Chiamiamo HEAPIFY-MAX da destra a sinistra sui tre indici:

$$\begin{aligned}[\,4, 10, 3, 5, 1, 8, 6\,] &\xrightarrow{i=2} [\,4, 10, 8, 5, 1, 3, 6\,] \\ &\xrightarrow{i=1} [\,4, 10, 8, 5, 1, 3, 6\,] \\ &\xrightarrow{i=0} [\,10, 5, 8, 4, 1, 3, 6\,].\end{aligned}$$

Attenzione: nell'ultimo passaggio il valore 4 scende prima sotto 10 e poi sotto 5, producendo $[10, 5, 8, 4, 1, 3, 6]$. La costruzione non ordina l'array; produce invece una disposizione dalla quale sarà economico consultare ed estrarre ripetutamente il massimo.

Proposizione 5.1 (Correttezza della costruzione dal basso)**.** La procedura COSTRUISCI-MAX-HEAP(A, n) trasforma qualunque array di n record in un max-heap contenente esattamente gli stessi record.

Dimostrazione. Le foglie, cioè i nodi con indice almeno $\lfloor n/2 \rfloor$, sono heap validi. Procediamo con induzione all'indietro sugli indici degli ultimi nodi interni. Prima di elaborare un indice i, tutti gli eventuali figli hanno indici maggiori di i e sono già radici di max-heap corretti. HEAPIFY-MAX ripristina quindi la proprietà nel sottoalbero di i. Dopo aver elaborato $i = 0$, la proprietà vale nell'intero albero. Le sole modifiche sono scambi di record, quindi il multinsieme degli elementi non cambia. □

5.8 La sorpresa: costruire un heap costa tempo lineare

A prima vista sembrerebbe che la costruzione esegua circa $n/2$ chiamate a una procedura che costa $O(\log n)$; ne ricaveremmo un limite $O(n \log n)$. Sarebbe corretto, ma molto pessimista. La maggior parte dei nodi interni è vicina alle foglie e non può percorrere molti livelli durante HEAPIFY-MAX. Soltanto pochissimi nodi, in prossimità della radice, possono scendere a lungo.

Teorema 5.2 (Costo di costruzione dal basso)**.** *La procedura COSTRUISCI-MAX-HEAP(A, n) richiede $\Theta(n)$ tempo nel caso peggiore per $n \geq 1$.*

Dimostrazione. In un heap binario il numero di nodi la cui altezza è h è al più $\lceil n/2^{h+1} \rceil$. Una chiamata a HEAPIFY-MAX su un nodo di altezza h esegue $O(h + 1)$ operazioni; per una foglia non viene effettuata alcuna chiamata nella costruzione. Pertanto, per costanti opportune e sommando sui possibili valori di h, il tempo è limitato superiormente da

$$O\left(\sum_{h \geq 1} \frac{n(h+1)}{2^h}\right) = O\left(n \sum_{h \geq 1} \frac{h+1}{2^h}\right) = O(n),$$

poiché la serie $\sum_{h \geq 1} (h + 1)/2^h$ converge. Gli arrotondamenti possono essere assorbiti nello stesso limite asintotico. D'altro canto, la procedura deve ricevere e trattare un input di n record; nel caso peggiore il costo è almeno $\Omega(n)$. Ne segue $\Theta(n)$. □

Il punto didattico merita di essere ricordato. *Una sequenza di chiamate costose nel caso peggiore può essere economica nel suo complesso, se la maggior parte delle chiamate lavora su problemi molto più piccoli*. Non basta conoscere il costo massimo di una singola chiamata: occorre capire su quali istanze essa viene realmente eseguita. Questa costruzione dal basso è generalmente associata alla formulazione e al perfezionamento degli algoritmi su heap di Robert W. Floyd nel 1964 [1, 3]. Per una trattazione algoritmica sistematica sono utili anche i capitoli dedicati alle strutture per selezione in Knuth [5].

6 Il min-heap: la stessa struttura, un ordine rovesciato

Finora abbiamo studiato nel dettaglio il max-heap, perché la scelta del record di chiave più alta è facile da visualizzare nell'esempio del pronto soccorso. Molti algoritmi, però, richiedono l'operazione opposta. Una simulazione vuole il prossimo evento nel tempo; un algoritmo sui grafi vuole la distanza minore; una procedura di codifica può voler combinare i due pesi più piccoli. In questi casi lavoriamo con un *min-heap*. La forma dell'albero e le formule sugli indici sono identiche; cambia soltanto il verso delle disuguaglianze.

Questo apparente dettaglio produce una piacevole economia progettuale. Non dobbiamo inventare altri algoritmi: basta comprendere il *principio* e tradurre in modo coerente tutti i confronti. Per evitare ambiguità riportiamo comunque lo pseudocodice completo delle operazioni fondamentali, in modo che sia possibile usarle senza rinviare continuamente alle procedure del max-heap.

6.1 Heapify-Min e la risalita

HEAPIFY-MIN assume che i sottoalberi dei figli siano già min-heap e controlla se il record nel nodo i sia più grande di quello di un figlio. Quando è necessario uno scambio, sceglie il *figlio più piccolo*: quel record deve salire affinché il padre diventi non maggiore di entrambi i figli. La procedura prosegue lungo il ramo in cui è sceso il vecchio valore.

Algoritmo 12 HEAPIFY-MIN(A, n, i): ripristino discendente in un min-heap

Dati: $0 \le i < n$; i sottoalberi dei figli di i sono min-heap

```
1:  mentre vero fai
2:      m ← i
3:      ℓ ← 2i + 1; r ← 2i + 2
4:      se ℓ < n e A[ℓ] < A[m] allora
5:          m ← ℓ
6:      fine se
7:      se r < n e A[r] < A[m] allora
8:          m ← r
9:      fine se
10:     se m = i allora
11:         restituisci
12:     fine se
13:     scambia(A[i], A[m])
14:     i ← m
15: fine mentre
```

Invece, se una nuova chiave è troppo piccola rispetto al proprio padre, la procedura corretta è una risalita. Come nel caso massimo, basta verificare un solo padre per iterazione.

Algoritmo 13 RISALI-MIN(A, i): riparazione verso la radice

```
1: mentre i > 0 fai
2:     p ← ⌊(i − 1)/2⌋
3:     se A[p] ≤ A[i] allora
4:         restituisci
5:     fine se
6:     scambia(A[p], A[i])
7:     i ← p
8: fine mentre
```

Il confronto $A[p] \leq A[i]$ ferma il percorso quando la proprietà di min-heap è soddisfatta. In caso di priorità uguali non spostiamo il record inutilmente: ciò riduce gli scambi, ma *non* rende automaticamente stabile l'heap rispetto all'ordine di inserimento.

6.2 Inserire un record ed estrarre il minimo

L'inserimento avviene ancora nell'ultima posizione disponibile dell'array. Se la chiave inserita è troppo piccola rispetto a quella del padre, RISALI-MIN corregge la violazione. Anche l'estrazione segue le stesse quattro azioni già viste: conservare la radice, spostare alla radice il record finale, ridurre la dimensione ed eseguire HEAPIFY-MIN.

Algoritmo 14 INSERISCI-MIN(A, n, x)

Dati: L'array ha una cella disponibile in posizione n

```
1: A[n] ← x
2: RISALI-MIN(A, n)
3: restituisci n + 1
```

Algoritmo 15 MINIMO(A, n) e ESTRAI-MIN(A, n)

```
1: procedura MINIMO(A, n)
2:     se n = 0 allora
3:         errore: heap vuoto
4:     fine se
5:     restituisci A[0]
6: fine procedura
7: procedura ESTRAI-MIN(A, n)
8:     se n = 0 allora
9:         errore: heap vuoto
10:    fine se
11:    x ← A[0]
12:    se n = 1 allora
13:        restituisci (x, 0)
14:    fine se
15:    A[0] ← A[n − 1]
16:    n ← n − 1
17:    HEAPIFY-MIN(A, n, 0)
18:    restituisci (x, n)
19: fine procedura
```

Seguiamo un'estrazione su un min-heap di nove valori:

$$A = [\, 2, 5, 4, 9, 7, 8, 6, 14, 11 \,].$$

L'operazione ESTRAI-MIN restituisce 2. Il record finale, di valore 11, viene trasferito alla radice, e la dimensione passa da 9 a 8. HEAPIFY-MIN confronta 11 con 5 e 4, scegliendo 4; poi confronta 11 con 8 e 6, scegliendo 6. La sequenza dei valori dell'heap è

$$\begin{aligned}[\, \mathbf{11}, 5, 4, 9, 7, 8, 6, 14 \,] &\longrightarrow [\, 4, 5, \mathbf{11}, 9, 7, 8, 6, 14 \,] \\ &\longrightarrow [\, 4, 5, 6, 9, 7, 8, \mathbf{11}, 14 \,].\end{aligned}$$

Il minimo successivo è dunque 4, nella prima cella. L'estrazione ha nuovamente visitato un solo cammino e costa $O(\log n)$ tempo.

6.3 Decrease-Key e Increase-Key nel min-heap

Nel min-heap una chiave che *diminuisce* può diventare più piccola del padre e deve risalire; una chiave che *aumenta* può diventare più grande dei figli e richiede HEAPIFY-MIN. Per evitare chiamate con segno incoerente controlliamo sempre che la nuova chiave rispetti la direzione dell'operazione.

Algoritmo 16 DIMINUISCI-CHIAVE-MIN(A, n, i, k)

Dati: $0 \le i < n$

```
1: se k > chiave(A[i]) allora
2:     errore: non è una diminuzione di chiave
3: fine se
4: chiave(A[i]) ← k
5: RISALI-MIN(A, i)
```

Algoritmo 17 AUMENTA-CHIAVE-MIN(A, n, i, k)

Dati: $0 \le i < n$

```
1: se k < chiave(A[i]) allora
2:     errore: non è un aumento di chiave
3: fine se
4: chiave(A[i]) ← k
5: HEAPIFY-MIN(A, n, i)
```

Riprendiamo il min-heap iniziale $[2, 5, 4, 9, 7, 8, 6, 14, 11]$. Se la chiave del nodo $A[8]$ scende da 11 a 1, il valore 1 risale attraverso i nodi con valori 9, 5 e 2; otteniamo

$$[\, 1, 2, 4, 5, 7, 8, 6, 14, 9 \,].$$

Se invece aumentassimo la chiave del nodo di indice 2 da 4 a 12, HEAPIFY-MIN scambierebbe 12 con il suo figlio più piccolo, 6, producendo $[2, 5, 6, 9, 7, 8, 12, 14, 11]$. Ancora una volta la scelta di *quale* procedura chiamare deriva direttamente dall'invariante, senza richiedere di ripensare l'intero algoritmo.

6.4 Eliminazione e costruzione per un min-heap

Per eliminare un record noto in posizione i sostituiamo quel record con l'ultimo. Se il sostituto risulta più piccolo del padre, risaliamo; in caso contrario applichiamo HEAPIFY-MIN sulla sua nuova posizione.

La precondizione di HEAPIFY-MIN è soddisfatta perché i sottoalberi sotto i non sono stati modificati, ad eccezione del valore del loro padre.

Algoritmo 18 ELIMINA-MIN(A, n, i): eliminazione per indice

Dati: $0 \le i < n$
1: $x \leftarrow A[i]$
2: **se** $i = n - 1$ **allora**
3: **restituisci** $(x, n - 1)$
4: **fine se**
5: $A[i] \leftarrow A[n - 1]$
6: $n \leftarrow n - 1$
7: **se** $i > 0$ **e** $A[i] < A[\lfloor (i - 1)/2 \rfloor]$ **allora**
8: RISALI-MIN(A, i)
9: **altrimenti**
10: HEAPIFY-MIN(A, n, i)
11: **fine se**
12: **restituisci** (x, n)

La costruzione dal basso è ancora più semplice: gli indici dei nodi interni sono gli stessi, e l'unica differenza consiste nel chiamare HEAPIFY-MIN anziché HEAPIFY-MAX. Proprio come nella versione massima, il tempo complessivo rimane lineare, nonostante la singola chiamata possa richiedere tempo logaritmico.

Algoritmo 19 COSTRUISCI-MIN-HEAP(A, n)

1: **per** $i \leftarrow \lfloor n/2 \rfloor - 1$ **fino a** 0, **decrescendo fai**
2: HEAPIFY-MIN(A, n, i)
3: **fine per**

6.5 Un'unica idea dietro due famiglie di procedure

Il codice completo evidenzia una simmetria utile anche per progettare librerie software: un heap può essere parametrizzato da una *relazione di priorità*. In un max-heap il padre deve precedere i figli secondo $\ge$; in un min-heap secondo $\le$. Una realizzazione generica può utilizzare un comparatore per stabilire quale dei due elementi abbia precedenza, anziché duplicare materialmente tutti gli algoritmi. Per finalità didattiche, però, le due versioni esplicite aiutano a evitare uno degli errori più comuni: applicare HEAPIFY quando la violazione deve invece propagarsi verso il padre, o viceversa.

7 Quanto costano davvero le operazioni?

Siamo ora in grado di confrontare le operazioni della coda con priorità in modo sistematico. In un heap con n elementi consultare la radice o conoscere il numero degli elementi costa tempo costante. Tutte le operazioni che comportano una risalita o una discesa lavorano lungo un cammino di lunghezza al più $\lfloor \log_2 n \rfloor$. La costruzione dal basso fa eccezione in senso positivo: pur chiamando ripetutamente HEAPIFY, richiede tempo lineare nel numero di record.

L'espressione $O(1)$ per lo spazio ausiliario presuppone che l'array abbia spazio sufficiente o che l'eventuale ridimensionamento venga contabilizzato a parte. Se l'array deve crescere e viene riallocato, un singolo inserimento può spendere $O(n)$ tempo e utilizzare temporaneamente $O(n)$ memoria per la

Tabella 4: Costi delle operazioni su heap binari. Le modifiche e la cancellazione per indice presuppongono di conoscere la posizione del record; lo spazio non include i record già memorizzati nell'array.

Operazione	Tempo peggiore	Spazio ausiliario	Osservazioni
MASSIMO/MINIMO	$O(1)$	$O(1)$	Nessuna rimozione
INSERISCI	$O(\log n)$	$O(1)$	Capacità disponibile
ESTRAI-MAX/ESTRAI-MIN	$O(\log n)$	$O(1)$	Radice e ultima foglia
INCREASE-KEY	$O(\log n)$	$O(1)$	Posizione nota
DECREASE-KEY	$O(\log n)$	$O(1)$	Posizione nota
ELIMINA	$O(\log n)$	$O(1)$	Posizione nota
HEAPIFY	$O(\log n)$	$O(1)$	Sottoalberi già heap
COSTRUISCI-HEAP	$\Theta(n)$	$O(1)$	Costruzione dal basso
DIMENSIONE/VUOTA	$O(1)$	$O(1)$	Si consulta n

copia; con una politica geometrica, gli inserimenti conservano il limite ammortizzato $O(\log n)$, oltre al costo di conservazione degli n record.

7.1 Un confronto con le liste, senza dimenticare le ipotesi

Nel caso di una lista non ordinata, trovare il massimo richiede in generale di esaminare molti record: l'informazione su quale sia la priorità estrema non è mantenuta. Una lista ordinata la conserva, ma deve modificare una parte potenzialmente grande della sequenza ogni volta che compare un nuovo elemento. L'heap fa una scelta diversa: conserva soltanto le relazioni tra padri e figli, e proprio questa rinuncia all'ordine totale permette di limitare il lavoro ai cammini dell'albero.

L'heap non domina qualsiasi struttura dati in ogni operazione. Per cercare una chiave arbitraria, ad esempio, un albero binario di ricerca bilanciato può essere preferibile. E per una singola estrazione su una collezione piccolissima, una scansione lineare potrebbe avere costanti inferiori. La struttura corretta dipende sempre dall'*insieme delle operazioni più frequenti*, non soltanto dalla complessità di una di esse.

7.2 Aggiornare un record conoscendone soltanto l'identificatore

In un algoritmo su grafi chiediamo spesso qualcosa come "diminuisci la distanza del vertice v". Se conosciamo soltanto il suo nome, come troviamo in quale posizione dell'heap sia memorizzato? Cercarlo linearmente vanificherebbe il vantaggio dell'operazione DIMINUISCI-CHIAVE-MIN. Una soluzione comune è mantenere una *mappa delle posizioni*: per ciascun identificatore v conserviamo $\mathrm{pos}[v]$, l'indice corrente in A.

Quando due record dell'heap si scambiano, aggiorniamo anche le due posizioni memorizzate. Questo costa tempo costante per scambio se gli identificatori indicizzano direttamente un array oppure se l'accesso alla mappa ha il costo previsto dall'implementazione scelta. In un algoritmo sui grafi i vertici numerati da 0 a $V - 1$ si prestano perfettamente a un array delle posizioni. Con un accesso $O(1)$, l'aggiornamento di una priorità nota per identificatore mantiene il costo $O(\log n)$.

È utile però distinguere l'heap *nudo*, che contiene soltanto i record, da una coda con priorità completa, che può includere anche una struttura ausiliaria per localizzarli. Nel primo caso le modifiche per indice sono logaritmiche; nel secondo la prestazione dipende anche dal meccanismo di localizzazione. Non è corretto nascondere una ricerca lineare dietro un'operazione dichiarata logaritmica.

7.3 Priorità uguali: ordine di arrivo e stabilità

L'heap binario non garantisce, di per sé, che due record con uguale priorità escano nel loro ordine d'inserimento. Gli scambi effettuati in seguito a estrazioni e inserimenti possono alterare l'ordine relativo dei record equivalenti. Se questo è richiesto dall'applicazione, possiamo affiancare alla priorità un contatore crescente di arrivo.

Per una coda che estrae il massimo, il record può avere chiave composta $(p, -t)$, dove p è la priorità e t è il contatore di inserimento: si confronta prima p e, soltanto a parità di priorità, si preferisce il valore più piccolo di t. Per una coda che estrae il minimo si può usare (p, t), sempre scegliendo come migliore la coppia lessicograficamente più piccola. Il confronto resta costante, a condizione che entrambe le componenti siano rappresentabili con un numero limitato di parole di macchina.

Il meccanismo illustra un principio che ricorre spesso: *se una proprietà importante non è garantita dalla struttura, talvolta la si può incorporare nella definizione della chiave*. Naturalmente bisogna scegliere con cura il contatore e gestirne gli eventuali limiti di rappresentazione.

8 Tre applicazioni che fanno emergere la struttura

8.1 Una simulazione: il futuro in una coda

Immaginiamo di simulare una piccola rete di sportelli. Il programma conosce diversi eventi futuri: una richiesta che arriverà al tempo 12, un'altra che terminerà al tempo 7 e una terza che comincerà al tempo 9. Non possiamo elaborarli secondo l'ordine in cui sono stati creati nella memoria: dobbiamo sempre scegliere quello con il *tempo simulato minimo*.

Inseriamo quindi tre record con chiavi temporali 12, 7 e 9 in un min-heap. Il primo ESTRAI-MIN restituisce l'evento del tempo 7. Durante la sua elaborazione potrebbe essere generato un nuovo evento al tempo 8: lo inseriamo nella stessa struttura. Il successivo ESTRAI-MIN restituirà l'evento del tempo 8, poi quello del tempo 9, e così via. Il programma non ha mai avuto bisogno di ordinare permanentemente tutti gli eventi futuri; ha soltanto mantenuto accessibile il prossimo.

Questo esempio è particolarmente utile perché rivela che una coda con priorità non è una semplice operazione di ordinamento una tantum. *Il contenuto della struttura continua a cambiare mentre il programma lavora*. L'interfaccia deve quindi rendere efficienti non solo le estrazioni, ma anche gli inserimenti che si alternano a esse.

8.2 Cammini minimi: quando una distanza migliora

Consideriamo una rete di luoghi con archi di costo non negativo. Nell'algoritmo di Dijkstra si mantiene per diversi vertici una distanza provvisoria dalla sorgente. La prossima scelta ricade sul vertice con la distanza provvisoria minima. Se scopriamo un cammino migliore per raggiungere un vertice ancora da elaborare, la sua priorità cambia: la nuova distanza è più piccola.

È proprio la combinazione ESTRAI-MIN e DIMINUISCI-CHIAVE-MIN che rende il min-heap particolarmente adatto a questa situazione. Non entreremo qui nella prova di correttezza dell'algoritmo sui grafi, ma possiamo già comprendere la funzione della struttura dati: scegliere il prossimo candidato e correggere rapidamente la posizione di un vertice quando la sua distanza migliora. Nel caso in cui ogni vertice abbia un identificatore numerico, mantenere un array delle posizioni evita di dover cercare ogni volta il record da aggiornare.

8.3 Huffman: costruire un codice scegliendo due minimi

Un altro esempio, già presente nella storia dell'informatica ben prima della pubblicazione dei lavori sugli heap binari, è quello della codifica di Huffman. Si parte da simboli dotati di frequenze e, ripetutamente, si scelgono i due pesi minimi per combinarli in un nuovo nodo di peso pari alla loro somma. La procedura continua finché rimane un solo nodo radice, che descrive l'albero di codifica.

Una min-priority queue consente di esprimere in modo naturale questa sequenza: due estrazioni minime, una somma e un inserimento. Non è necessario usare il binary heap per definire l'algoritmo di Huffman, né è storicamente corretto suggerire che nel 1952 fosse già stata proposta la moderna struttura dell'heap binario. Ciò che l'esempio mette in risalto è la separazione tra l'*idea algoritmica*, cioè selezionare ripetutamente due elementi minimi, e la *struttura dati* scelta per renderla efficiente.

9 Una breve storia: l'idea, la forma, l'efficienza

Il problema di scegliere più volte una priorità estrema precede la descrizione dell'heap binario. David A. Huffman pubblicò nel 1952 il suo metodo per i codici a ridondanza minima; Edsger W. Dijkstra presentò nel 1959 il celebre lavoro sui cammini minimi [2, 4]. Le idee di scelta per priorità erano dunque già presenti in problemi profondamente diversi: compressione dell'informazione e ricerca di percorsi.

Nel 1964 *Communications of the ACM* pubblicò *Algorithm 232: Heapsort* di J. W. J. Williams, associato all'introduzione del binary heap come struttura implicita per gestire efficacemente la selezione di elementi estremi. Nello stesso anno Robert W. Floyd pubblicò *Algorithm 245: Treesort 3*, che sviluppò e perfezionò la tecnica di organizzazione e manipolazione dell'heap [3, 8]. Le date, vicinissime, raccontano un momento significativo: nel giro di pochi mesi una struttura concettualmente semplice divenne il nucleo di un metodo algoritmico solido e studiabile con precisione.

La costruzione dal basso in tempo lineare è una parte essenziale di questa storia. L'intuizione ingenua di inserire ripetutamente gli elementi dà già una buona complessità, ma un'analisi più attenta della geometria dell'albero mostra che si può fare meglio. Una volta compresa questa idea, l'heap appare meno come un insieme di formule da memorizzare e più come un caso esemplare di *progettazione a partire dalle invarianti*.

Nella dispensa sugli algoritmi di ordinamento abbiamo già incontrato Heapsort, che utilizza questa struttura per estrarre ripetutamente un elemento estremo. Qui abbiamo deliberatamente mantenuto il punto di vista opposto: invece di usare l'heap per ordinare una sequenza, abbiamo studiato la struttura come realizzazione autonoma e dinamica del tipo di dato astratto *coda con priorità*. È una distinzione importante: la struttura resta utile anche quando non abbiamo nessuna intenzione di ordinare tutti gli elementi.

10 Conclusioni: quanto ordine serve davvero?

Siamo partiti da una situazione nella quale l'ordine di arrivo non basta. La soluzione non è stata ordinare continuamente tutti i record, ma costruire una struttura capace di mantenere un'informazione più mirata: *l'elemento di priorità estrema si trova sempre alla radice*. Questa promessa è sostenuta da due proprietà indipendenti: la forma completa dell'albero e il vincolo locale tra padre e figli.

Le operazioni di aggiornamento conservano la promessa lavorando su un solo cammino. L'inserimento colloca prima il nodo dove lo impone la forma, poi fa risalire il record se necessario. L'estrazione usa l'ultima foglia per non lasciare buchi e chiama HEAPIFY per ripristinare la priorità. Le modifiche della chiave scelgono fra risalita e HEAPIFY in base alla direzione della possibile violazione. E la costruzione dal

basso mostra che, quando i sottoproblemi sono quasi tutti piccoli, un insieme di operazioni logaritmiche può richiedere in totale soltanto tempo lineare.

Il passaggio dall'albero all'array ha aggiunto un'ultima sorpresa: una struttura che avevamo immaginato fatta di nodi e collegamenti può essere conservata come una sequenza compatta di record. Le formule $2i + 1$, $2i + 2$ e $\lfloor (i - 1)/2 \rfloor$ bastano a ricostruire le relazioni tra padre e figli; le operazioni bitwise ne rendono esplicita la base binaria. Non abbiamo semplificato soltanto l'implementazione: abbiamo visto come la *forma matematica di una struttura dati* possa suggerire la sua rappresentazione più efficace.

Questa è la lezione generale dell'heap: per risolvere efficientemente un problema dinamico non è sempre necessario conservare un ordine totale. Talvolta basta un ordine locale accuratamente scelto, unito alla capacità di capire in quale direzione una modifica possa propagare l'unica violazione dell'invariante.

Riferimenti bibliografici

[1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 4 edition, 2022. ISBN 9780262046305.

[2] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959. doi: 10.1007/BF01386390.

[3] Robert W. Floyd. Algorithm 245: Treesort 3. *Communications of the ACM*, 7(12):701, 1964. doi: 10.1145/355588.365103.

[4] David A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098–1101, 1952. doi: 10.1109/JRPROC.1952.273898.

[5] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2 edition, 1998. ISBN 9780201896855.

[6] Hiroyuki Nakao, Isao Ukai, and Joji Kotani. A review of the history of the origin of triage from a disaster medicine perspective. *Acute Medicine & Surgery*, 4(4):379–384, 2017. doi: 10.1002/ams2.293.

[7] Iain Robertson-Steel. Evolution of triage systems. *Emergency Medicine Journal*, 23(2):154–155, 2006. doi: 10.1136/emj.2005.030270.

[8] J. W. J. Williams. Algorithm 232: Heapsort. *Communications of the ACM*, 7(6):347–348, 1964. doi: 10.1145/512274.512284.